

AI Support Ticket Resolution Copilot

Cohort B · Interview Preparation Phase (IRP)

New Age Software Engineering Program · 8 Sessions · 2 Hours Each

8 Sessions	2 Hours Duration per Session	Backend Build Format	FastAPI + LangGraph Stack	SQLite / PostgreSQL Database	Deployed API Deliverable
---------------	------------------------------------	----------------------------	---------------------------------	------------------------------------	-----------------------------

Target Audience

Students who completed the 6–7 month NASE program covering Python, FastAPI, SQL, JWT auth, RAG, LangGraph, and agentic workflows. They have hands-on coding experience and use AI coding tools (Antigravity) to build features. Strong Python background; web dev experience optional.

Program End Goal

By the end of 8 sessions, students will have built a production-grade AI-powered backend — a FastAPI service that automatically classifies support tickets, retrieves grounded knowledge using RAG (ChromaDB + embeddings), resolves tickets through a LangGraph multi-node agent workflow, and includes auth, evals, guardrails, and deployment. Students will be able to explain every architectural decision, trade-off, and AI component in a technical interview.

Session-by-Session Breakdown

Session 1 Lay the Foundation: Build a Production FastAPI Backend

- FastAPI + APIRouter
- Pydantic models for request validation
- HTTP status codes (201, 200, 204, 404)
- In-memory storage
- Swagger UI at /docs

Deliverable: FastAPI app with 5 REST endpoints: POST/GET/PATCH/DELETE /tickets, all testable via Swagger UI

Session 2 **Make Data Stick: Add a Real Database Layer**

- SQLAlchemy ORM (SQLAlchemy + Pydantic)
- SQLite file-based persistence
- FastAPI dependency injection (get_session)
- CRUD DB operations
- Alembic first migration (stretch)

Deliverable: All 5 endpoints persisting to tickets.db; data survives server restarts

Session 3 **Lock It Down: JWT Auth + Role-Based Access Control**

- JWT auth with python-jose
- bcrypt password hashing
- /auth/signup and /auth/login endpoints
- get_current_user dependency
- Role-scoped data access (user vs. admin)
- 401 and 403 response patterns

Deliverable: Working auth system with JWT-protected routes, role-based guards, and unit tests for auth flows

Session 4 **Teach the Machine: Add an LLM Ticket Classifier**

- Gemini 1.5 Flash via google-generativeai
- JSON mode (response_mime_type="application/json")
- Low-temperature deterministic LLM calls
- Graceful fallback if LLM fails
- pytest mocking with unittest.mock.patch
- Ollama as local alternative

Deliverable: llm_classifier.py service + TicketClassification DB table; POST /tickets response includes category, priority, sentiment, urgency_score, summary, suggested_team

Session 5 **Ground the AI: Build a RAG Knowledge Base**

- RAG: retrieval vs. generation
- Text chunking by paragraph
- sentence-transformers all-MiniLM-L6-v2 (384-d, local)
- ChromaDB local persistent collection
- Top-3 cosine similarity retrieval
- Context injection into LLM prompt

Deliverable: GET /tickets/{id}/suggested-response endpoint returning suggested_response and sources; rag_service.py + ChromaDB persisted to disk

Session 6

Build the Brain: Add a LangGraph Agent Workflow

- LangGraph StateGraph with TypedDict state
- 4 nodes: classify → retrieve → generate → confidence_router
- Conditional edges on confidence_score < 0.7
- Human-in-the-loop flag (needs_human_review)
- Reuse of Sessions 4 and 5 functions

Deliverable: POST /tickets/{id}/resolve endpoint returning suggested_response, confidence_score, needs_human_review, classification; workflow in app/graph/nodes.py

Session 7

Bulletproof It: Evals, Guardrails, and Tests

- pytest with FastAPI TestClient and fixtures
- In-memory SQLite test DB
- Custom groundedness evaluator (string overlap or cosine)
- System prompt guardrails (block PII leakage, off-topic)
- Fallback response pattern
- confidence_score from ChromaDB similarity

Deliverable: tests/test_tickets.py (5–8 CRUD tests), tests/test_classifier.py, app/evals/groundedness.py, guardrail logic in generate_node

Session 8

Ship and Shine: Deploy, Demo, Ace the Interview

- requirements.txt pinning via pip freeze
- .env.example documentation
- README.md with Mermaid architecture diagram
- Railway / Render free-tier deployment
- Swagger UI verification on live URL
- 3-minute demo script + system design Q&A;

Deliverable: Live deployed backend on Railway/Render with Swagger UI; complete README; rehearsed 3-minute demo and system design explanation

Quick-Reference Summary Table

#	Session Title	Key Topics	Deliverable
1	Lay the Foundation: Build a Production FastAPI Backend	FastAPI + APIRouter · Pydantic models for request validation · HTTP status codes (201, 200, 204, 404)...	FastAPI app with 5 REST endpoints: POST/GET/PATCH/DELETE /tickets, all testable via Swagger UI
2	Make Data Stick: Add a Real Database Layer	SQLModel ORM (SQLAlchemy + Pydantic) · SQLite file-based persistence · FastAPI dependency injection (get_session)...	All 5 endpoints persisting to tickets.db; data survives server restarts
3	Lock It Down: JWT Auth + Role-Based Access Control	JWT auth with python-jose · bcrypt password hashing · /auth/signup and /auth/login endpoints...	Working auth system with JWT-protected routes, role-based guards, and unit tests for auth flows
4	Teach the Machine: Add an LLM Ticket Classifier	Gemini 1.5 Flash via google-generativeai · JSON mode (response_mime_type="application/json") · Low-temperature deterministic LLM calls...	llm_classifier.py service + TicketClassification DB table; POST /tickets response includes category, priority, sentiment, urgency_score, summary, suggested_team
5	Ground the AI: Build a RAG Knowledge Base	RAG: retrieval vs. generation · Text chunking by paragraph · sentence-transformers all-MiniLM-L6-v2 (384-d, local)...	GET /tickets/{id}/suggested-response endpoint returning suggested_response and sources; rag_service.py + ChromaDB persisted to disk
6	Build the Brain: Add a LangGraph Agent Workflow	LangGraph StateGraph with TypedDict state · 4 nodes: classify → retrieve → generate → confidence_router · Conditional edges on confidence_score < 0.7...	POST /tickets/{id}/resolve endpoint returning suggested_response, confidence_score, needs_human_review, classification; workflow in app/graph/nodes.py
7	Bulletproof It: Evals, Guardrails, and Tests	pytest with FastAPI TestClient and fixtures · In-memory SQLite test DB · Custom groundedness evaluator (string overlap or cosine)...	tests/test_tickets.py (5–8 CRUD tests), tests/test_classifier.py, app/evals/groundedness.py, guardrail logic in generate_node
8	Ship and Shine: Deploy, Demo, Ace the Interview	requirements.txt pinning via pip freeze · .env.example documentation · README.md with Mermaid architecture diagram...	Live deployed backend on Railway/Render with Swagger UI; complete README; rehearsed 3-minute demo and system design explanation